Experiment No. 3
Train a Deep Convolution Generative Multi-Layer (DCGAN) Network Model for MNIST dataset.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Train a Deep Convolution Generative Multi-Layer (DCGAN) Network Model for MNIST dataset

Objective:

• To understand the concept of Generative Adversarial Networks (GANs).
• To implement Deep Convolutional GAN (DCGAN) for image generation.
• To train a model on MNIST dataset for generating handwritten digits.
• To analyze the performance of generator and discriminator networks.

Theory:

Generative Adversarial Network (GAN):
GAN is a deep learning model consisting of two neural networks:
Generator: Generates fake data (images)
Discriminator: Distinguishes between real and fake data
Both networks are trained simultaneously in a competitive manner.

Deep Convolutional GAN (DCGAN):

DCGAN is an extension of GAN that uses Convolutional Neural Networks (CNNs) instead of fully connected layers.

Key Features of DCGAN:

- Uses convolutional layers for image processing
- Uses batch normalization
- Uses ReLU (Generator) and LeakyReLU (Discriminator)
- Removes pooling layers

Working:

1. Generator creates fake images from random noise
2. Discriminator evaluates images (real or fake)
3. Both networks improve through training
4. Generator learns to create realistic images

Implementation:

• Import required libraries such as TensorFlow/Keras, NumPy, and Matplotlib.
• Load and preprocess MNIST dataset (normalize images).
• Define the Generator network using convolutional layers.
• Define the Discriminator network using CNN layers.
• Combine both networks to form the DCGAN model.
• Train the model using:
  - Adversarial training process
  - Binary Cross-Entropy loss function
• Generate new images using trained Generator.
• Visualize generated images.

Code:

```
# Install dependencies:

# pip install tensorflow matplotlib numpy


import tensorflow as tf

from tensorflow.keras import layers

import numpy as np

import matplotlib.pyplot as plt


# Step 1: Load and preprocess MNIST dataset

(x_train, _), (_, _) = tf.keras.datasets.mnist.load_data()


x_train = x_train.astype('float32')

x_train = (x_train - 127.5) / 127.5   # Normalize to [-1, 1]
```

```
x_train = np.expand_dims(x_train, axis=-1)


BUFFER_SIZE = 60000

BATCH_SIZE = 256


train_dataset = tf.data.Dataset.from_tensor_slices(x_train)\

    .shuffle(BUFFER_SIZE).batch(BATCH_SIZE)


# Step 2: Build Generator

def build_generator():

    model = tf.keras.Sequential()


    model.add(layers.Dense(7*7*256, use_bias=False, input_shape=(100,)))

    model.add(layers.BatchNormalization())

    model.add(layers.ReLU())


    model.add(layers.Reshape((7, 7, 256)))


    model.add(layers.Conv2DTranspose(128, (5,5), strides=(1,1), padding='same',
use_bias=False))

    model.add(layers.BatchNormalization())

    model.add(layers.ReLU())
```

```
    model.add(layers.Conv2DTranspose(64, (5,5), strides=(2,2), padding='same',
use_bias=False))

    model.add(layers.BatchNormalization())

    model.add(layers.ReLU())



    model.add(layers.Conv2DTranspose(1, (5,5), strides=(2,2), padding='same',
use_bias=False, activation='tanh'))



    return model



# Step 3: Build Discriminator

def build_discriminator():

    model = tf.keras.Sequential()



    model.add(layers.Conv2D(64, (5,5), strides=(2,2), padding='same',
input_shape=[28,28,1]))

    model.add(layers.LeakyReLU())

    model.add(layers.Dropout(0.3))



    model.add(layers.Conv2D(128, (5,5), strides=(2,2), padding='same'))

    model.add(layers.LeakyReLU())

    model.add(layers.Dropout(0.3))



    model.add(layers.Flatten())
```

```
    model.add(layers.Dense(1))

    return model

generator = build_generator()
discriminator = build_discriminator()

# Step 4: Loss and Optimizers
cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    return real_loss + fake_loss

generator_optimizer = tf.keras.optimizers.Adam(1e-4)
discriminator_optimizer = tf.keras.optimizers.Adam(1e-4)

# Step 5: Training
```

```
EPOCHS = 20

noise_dim = 100

num_examples_to_generate = 16


seed = tf.random.normal([num_examples_to_generate, noise_dim])


@tf.function

def train_step(images):

    noise = tf.random.normal([BATCH_SIZE, noise_dim])


    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:

        generated_images = generator(noise, training=True)


        real_output = discriminator(images, training=True)

        fake_output = discriminator(generated_images, training=True)


        gen_loss = generator_loss(fake_output)

        disc_loss = discriminator_loss(real_output, fake_output)


    gradients_of_generator = gen_tape.gradient(gen_loss, generator.trainable_variables)

    gradients_of_discriminator = disc_tape.gradient(disc_loss,
discriminator.trainable_variables)
```

```
    generator_optimizer.apply_gradients(zip(gradients_of_generator,
generator.trainable_variables))

    discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator,
discriminator.trainable_variables))



def train(dataset, epochs):

    for epoch in range(epochs):

        for image_batch in dataset:

            train_step(image_batch)



        print(f"Epoch {epoch+1} completed")



        generate_and_save_images(generator, epoch + 1, seed)



# Step 6: Generate and display images

def generate_and_save_images(model, epoch, test_input):

    predictions = model(test_input, training=False)



    plt.figure(figsize=(4,4))



    for i in range(predictions.shape[0]):

        plt.subplot(4,4,i+1)

        plt.imshow(predictions[i, :, :, 0] * 127.5 + 127.5, cmap='gray')
```

```
    plt.axis('off')

    plt.suptitle(f"Epoch {epoch}")

    plt.show()


# Step 7: Train Model

train(train_dataset, EPOCHS)
```

Conclusion:

DCGAN successfully generates realistic handwritten digit images from the MNIST dataset. It demonstrates the power of generative models in deep learning and their applications in image synthesis.